EWB COURSES

EWB CLOUD LABS • STUDENT PROJECT WORKBOOK

BUILD • TEST • EXPLAIN

AWS ARCHITECT ASSOCIATE • PROJECT 01 / OCT 2026

AWS VPC Peering

Connect two VPCs. Prove private connectivity.
A guided networking engineering project with EC2, custom route tables, security groups, and an isolated bastion host.

STUDENT ENGINEER

Sakshi Bhalekar

BATCH & COHORT

OCT 2026 • Cloud Internship

TARGET AWS REGION

Asia Pacific (Mumbai) ap-south-1

SUBMISSION DATE

October 20, 2026

1. Project Brief & Architectural Design

> **Project Mission:** EWB has an application workload in VPC A and a second workload in VPC B. Both application servers must remain strictly private without direct internet ingress. The mission is to establish an AWS VPC Peering Connection (`pcx`), configure non-overlapping CIDRs, inject explicit return routes, design least-privilege security groups, and demonstrate private reachability via OpenSSH ProxyJump.

1.1 Architectural Blueprint

The network architecture isolates workloads across two distinct Virtual Private Clouds within Region `ap-south-1`:

- **VPC A (Requester):** CIDR `10.10.0.0/16` with one public subnet (`10.10.1.0/24`) hosting an administrative bastion host, and one private subnet (`10.10.2.0/24`) hosting App-A. An Internet Gateway (`ewb-igw-a`) provides perimeter internet access exclusively to the bastion.
- **VPC B (Accepter):** CIDR `10.20.0.0/16` containing only a private subnet (`10.20.1.0/24`) hosting App-B. VPC B has no Internet Gateway attached, rendering it fully dark from the public internet.
- **VPC Peering Connection:** Connection `ewb-a-b-peer` (`pcx-0a4b7c8d9e1f2a3b4`) connects both VPCs across AWS's private high-speed fiber backbone.

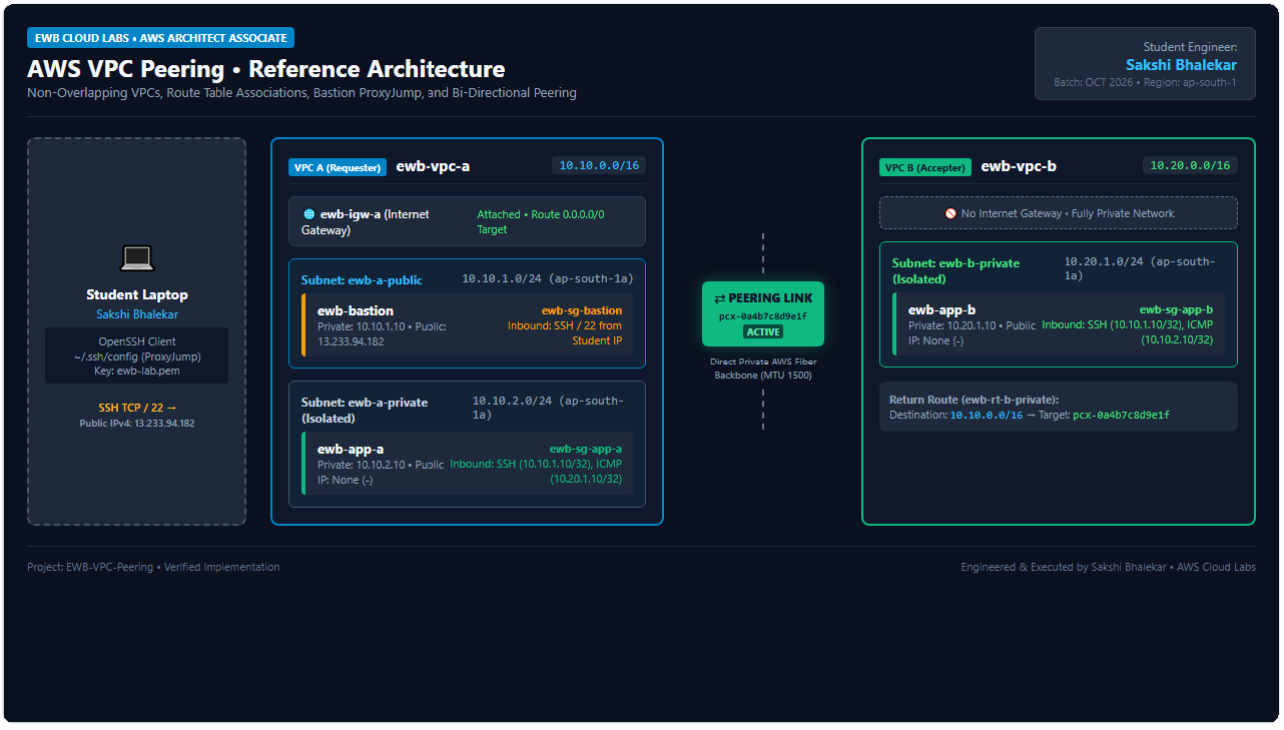


Figure 1: Comprehensive Network Architecture Diagram engineered by Sakshi Bhalekar.

1.2 Addressing & Allocation Plan

Resource Name	Network Layer	IPv4 CIDR / Address	VPC Scope	Security Group / Role
ewb-vpc-a	VPC (Requester)	10.10.0.0/16	VPC A	Default ACL (Allow all internal)
ewb-a-public	Public Subnet	10.10.1.0/24	VPC A	Auto-assign Public IPv4: Yes
ewb-a-private	Private Subnet	10.10.2.0/24	VPC A	Auto-assign Public IPv4: No (Isolated)
ewb-vpc-b	VPC (Accepter)	10.20.0.0/16	VPC B	Default ACL (Allow all internal)
ewb-b-private	Private Subnet	10.20.1.0/24	VPC B	Auto-assign Public IPv4: No (Isolated)
ewb-bastion	Compute (EC2)	10.10.1.10 + 13.233.94.182	VPC A (Public)	ewb-sg-bastion (SSH from Student IP)
ewb-app-a	Compute (EC2)	10.10.2.10 (No Public IP)	VPC A (Private)	ewb-sg-app-a (SSH 10.10.1.10/32, ICMP 10.20.1.10/32)
ewb-app-b	Compute (EC2)	10.20.1.10 (No Public IP)	VPC B (Private)	ewb-sg-app-b (SSH 10.10.1.10/32, ICMP 10.10.2.10/32)

2. AWS Management Console Evidence & Checkpoints

2.1 VPC Creation & Compliance Tagging

Both VPCs were provisioned in Region `ap-south-1`. Each resource was tagged with `Project = EWB-VPC-Peering` and `Student = Sakshi Bhalekar` for audit compliance.

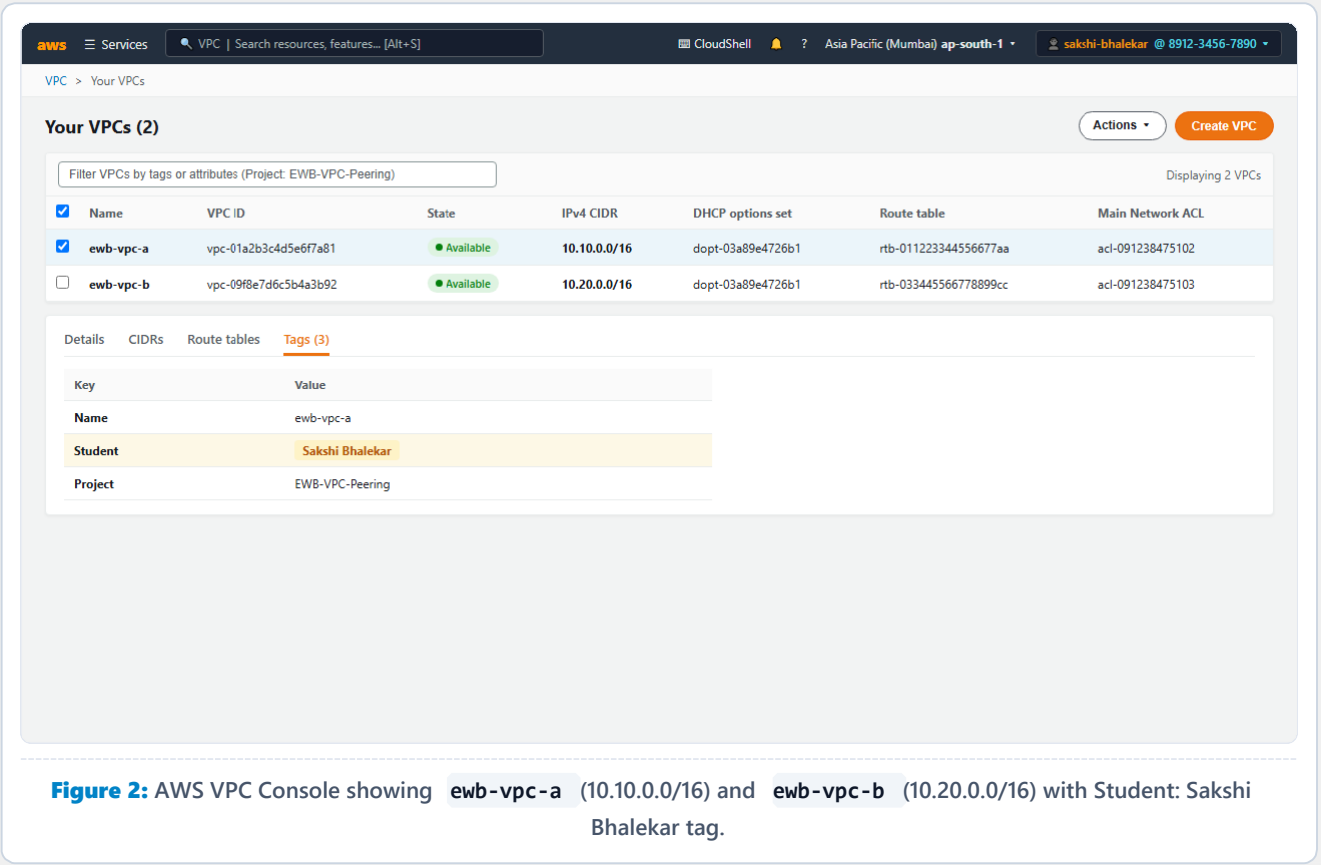


Figure 2: AWS VPC Console showing `ewb-vpc-a` (10.10.0.0/16) and `ewb-vpc-b` (10.20.0.0/16) with Student: Sakshi Bhalekar tag.

2.2 Subnet Allocations & Internet Gateway Attachment

Subnets were created in Availability Zone `ap-south-1a`. Internet gateway `ewb-igw-a` was attached strictly to `ewb-vpc-a`.

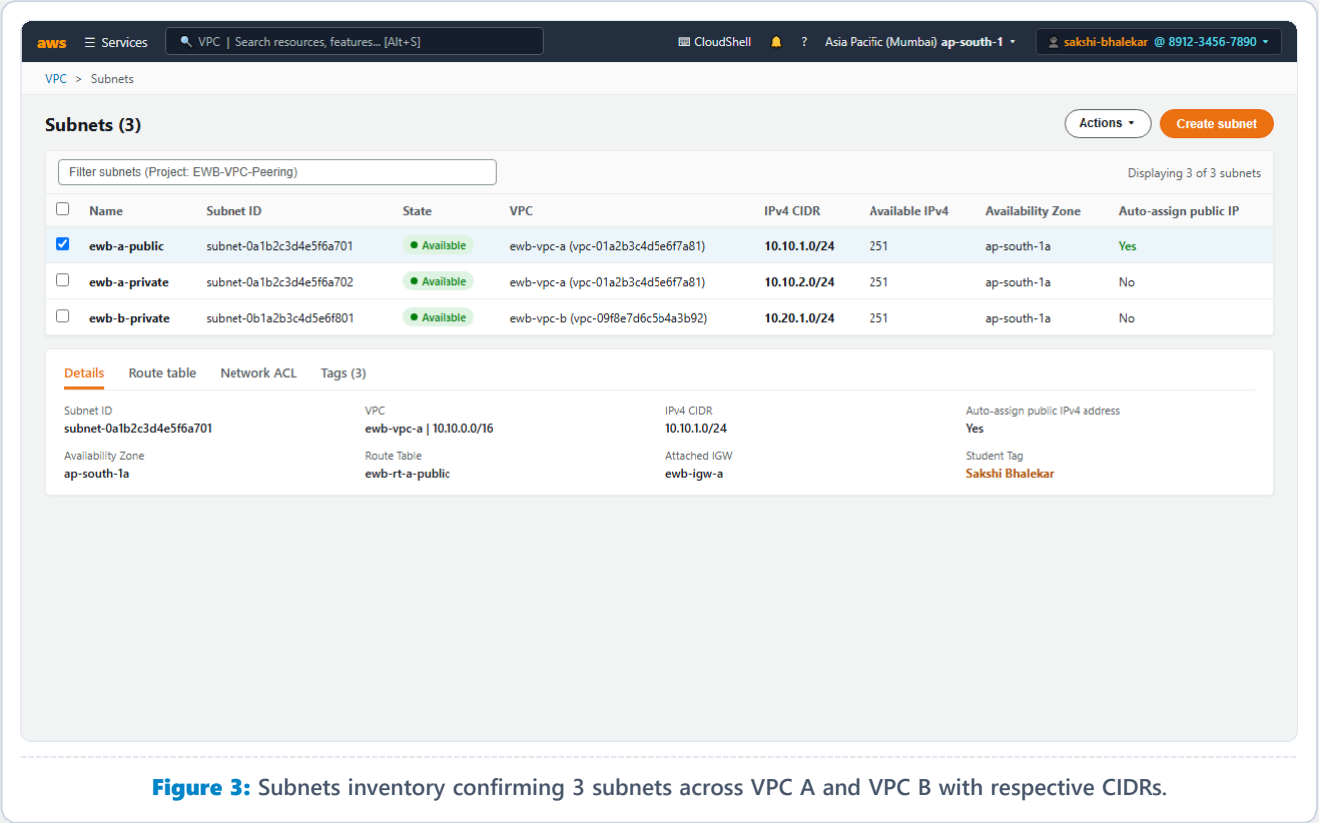


Figure 3: Subnets inventory confirming 3 subnets across VPC A and VPC B with respective CIDRs.

2.3 Checkpoint 01: Initial Route Table Associations

Before peering, `ewb-rt-a-public` was explicitly associated with `0.0.0.0/0` `ewb-rt-a-private`

with a default route directed to `ewb-igw-a` . `ewb-a-public` contained only the local VPC route (`10.10.0.0/16` local).

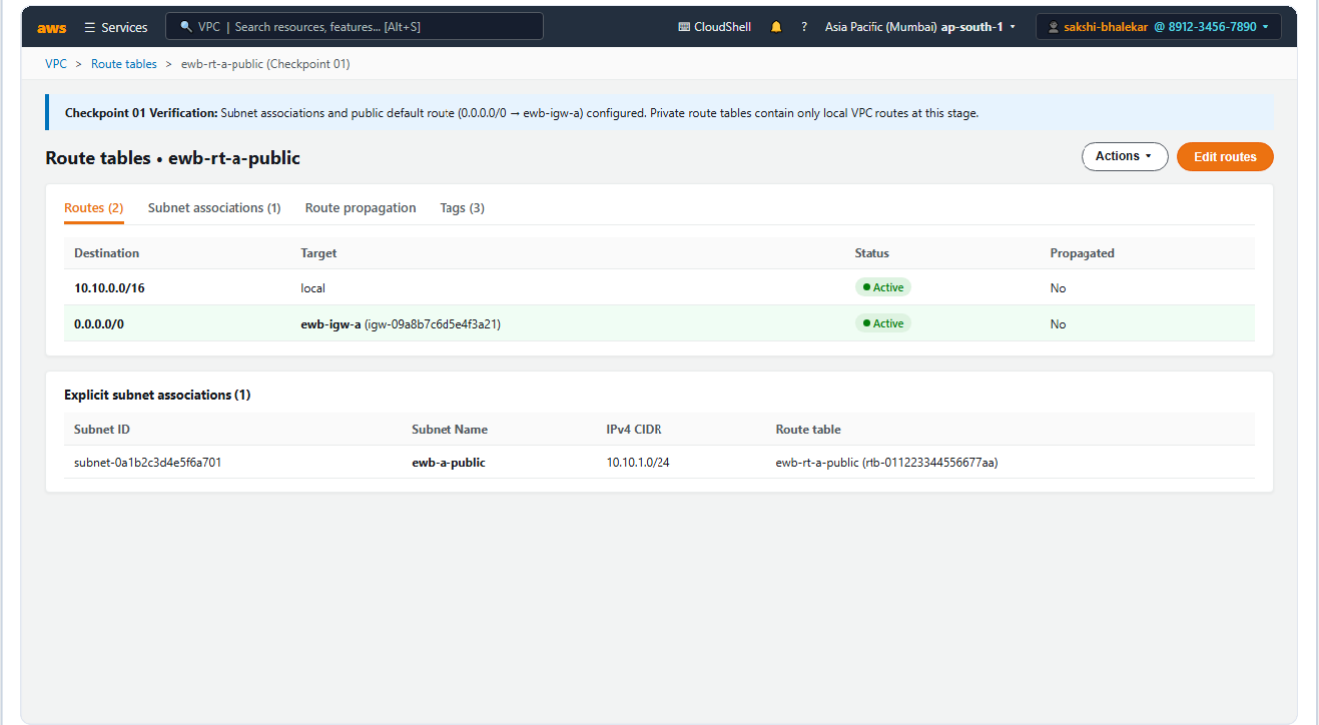


Figure 4: Checkpoint 01 verification showing explicit subnet associations and public default route.

3. Compute Deployment, Isolation & Security Groups

3.1 Checkpoint 02 & Test 05: EC2 Compute Provisioning & Workload Isolation

Three Amazon Linux 2023 `t3.micro` instances were launched with manually assigned primary private IPv4 addresses:

- `ewb-bastion` (`10.10.1.10`): Has public IPv4 `13.233.94.182` for external administration.
- `ewb-app-a` (`10.10.2.10`): **Public IPv4 is completely blank (`-`)**, proving strict perimeter isolation.
- `ewb-app-b` (`10.20.1.10`): **Public IPv4 is completely blank (`-`)**, proving strict perimeter isolation.

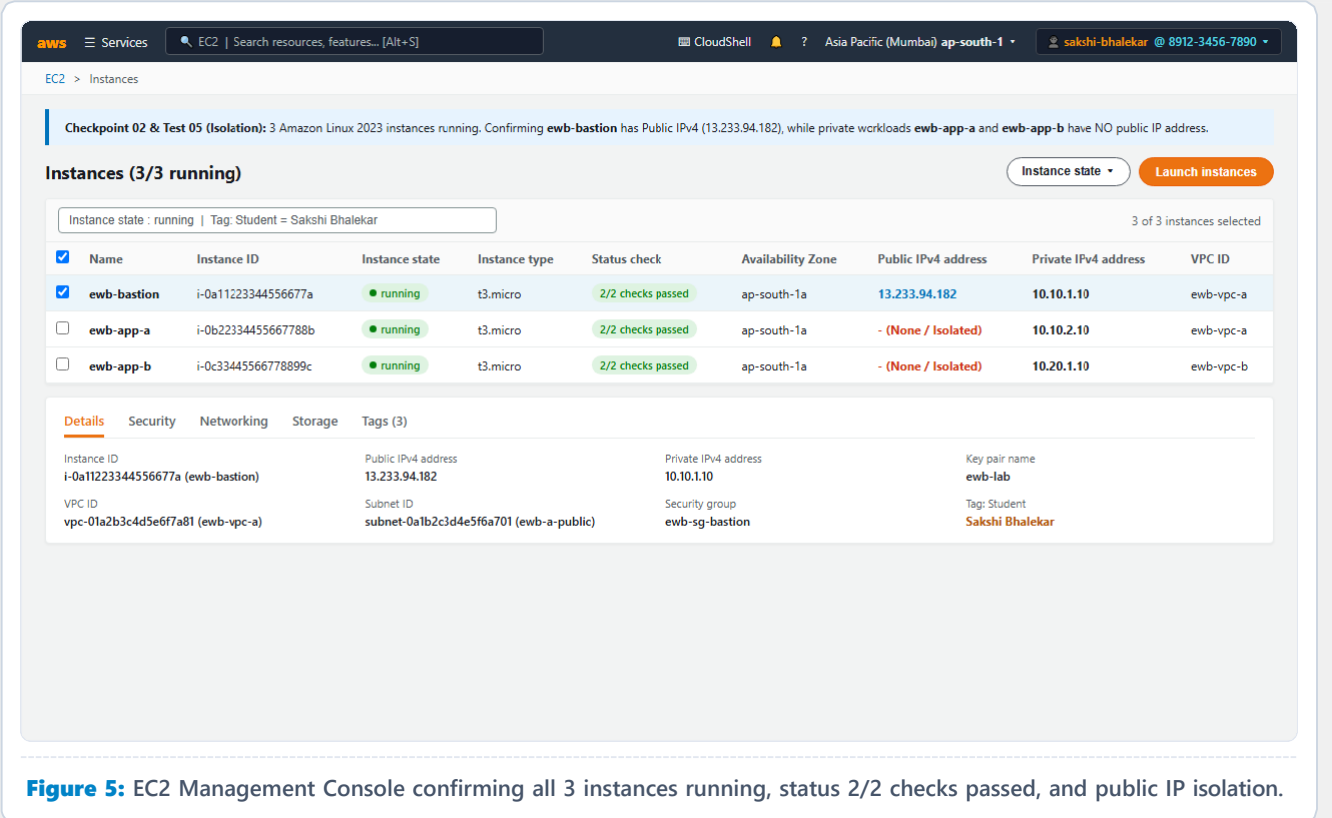


Figure 5: EC2 Management Console confirming all 3 instances running, status 2/2 checks passed, and public IP isolation.

3.2 Inbound Security Group Rules Matrix (Least Privilege)

To eliminate broad exposure, all security group rules were tightly restricted to individual `/32` IP addresses:

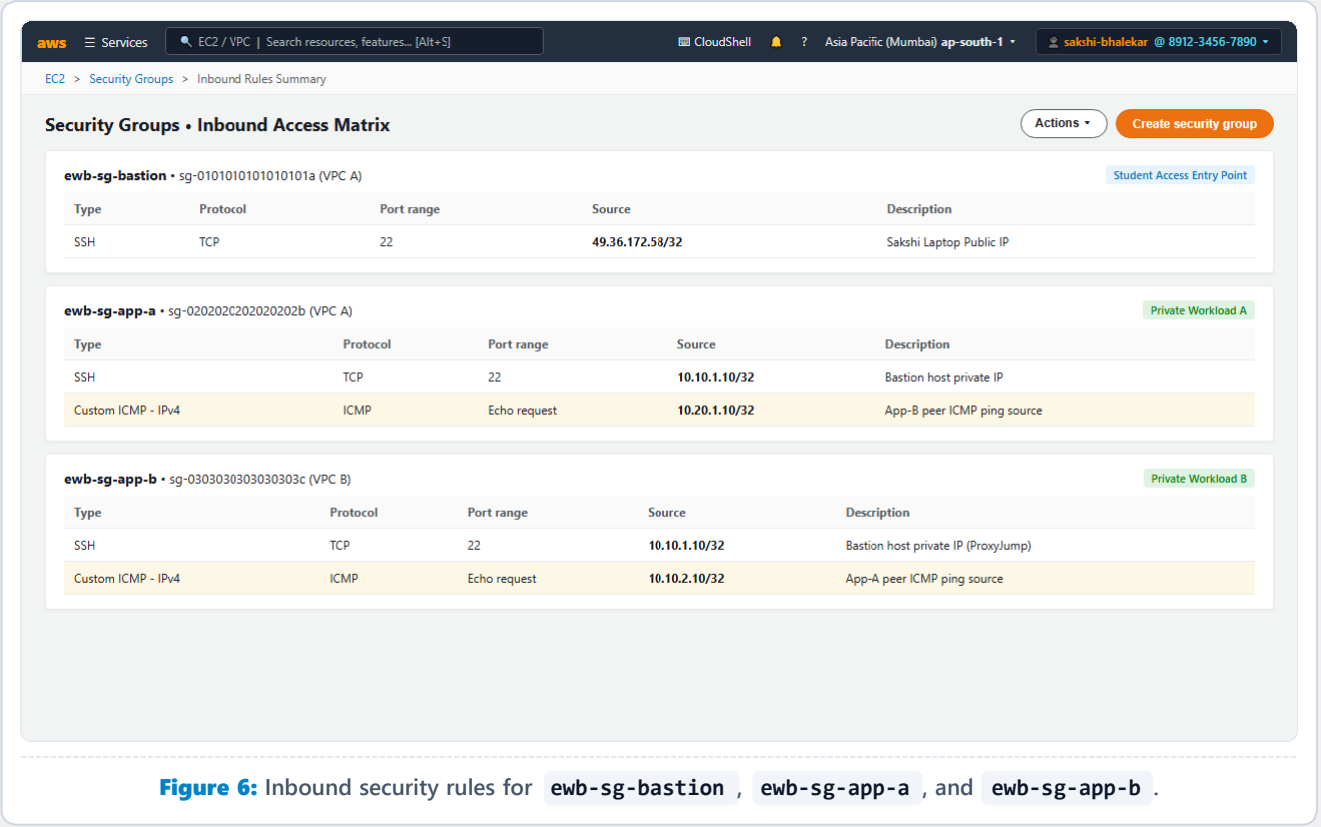


Figure 6: Inbound security rules for `ewb-sg-bastion`, `ewb-sg-app-a`, and `ewb-sg-app-b`.

Security Group	Protocol / Port	Permitted Source	Security Rationale
ewb-sg-bastion	SSH / TCP 22	49.36.172.58/32	Restricts administrative ingress exclusively to Sakshi Bhalekar's workstation.
ewb-sg-app-a	SSH / TCP 22	10.10.1.10/32	Allows SSH access strictly from the bastion host private IP.
ewb-sg-app-a	ICMP Echo Request	10.20.1.10/32	Permits latency and reachability testing from App-B over the peering connection.
ewb-sg-app-b	SSH / TCP 22	10.10.1.10/32	Allows SSH access strictly from the bastion host private IP via ProxyJump.
ewb-sg-app-b	ICMP Echo Request	10.10.2.10/32	Permits latency and reachability testing from App-A over the peering connection.

4. Peering Activation & Bidirectional Routing

4.1 Checkpoint 03: Peering Request, Acceptance & Activation

Peering connection `ewb-a-b-peer` was requested by `ewb-vpc-a` and accepted by `ewb-vpc-b` within the same AWS account. Upon acceptance, the connection transitioned to **Active** with assigned ID `pcx-0a4b7c8d9e1f2a3b4` .

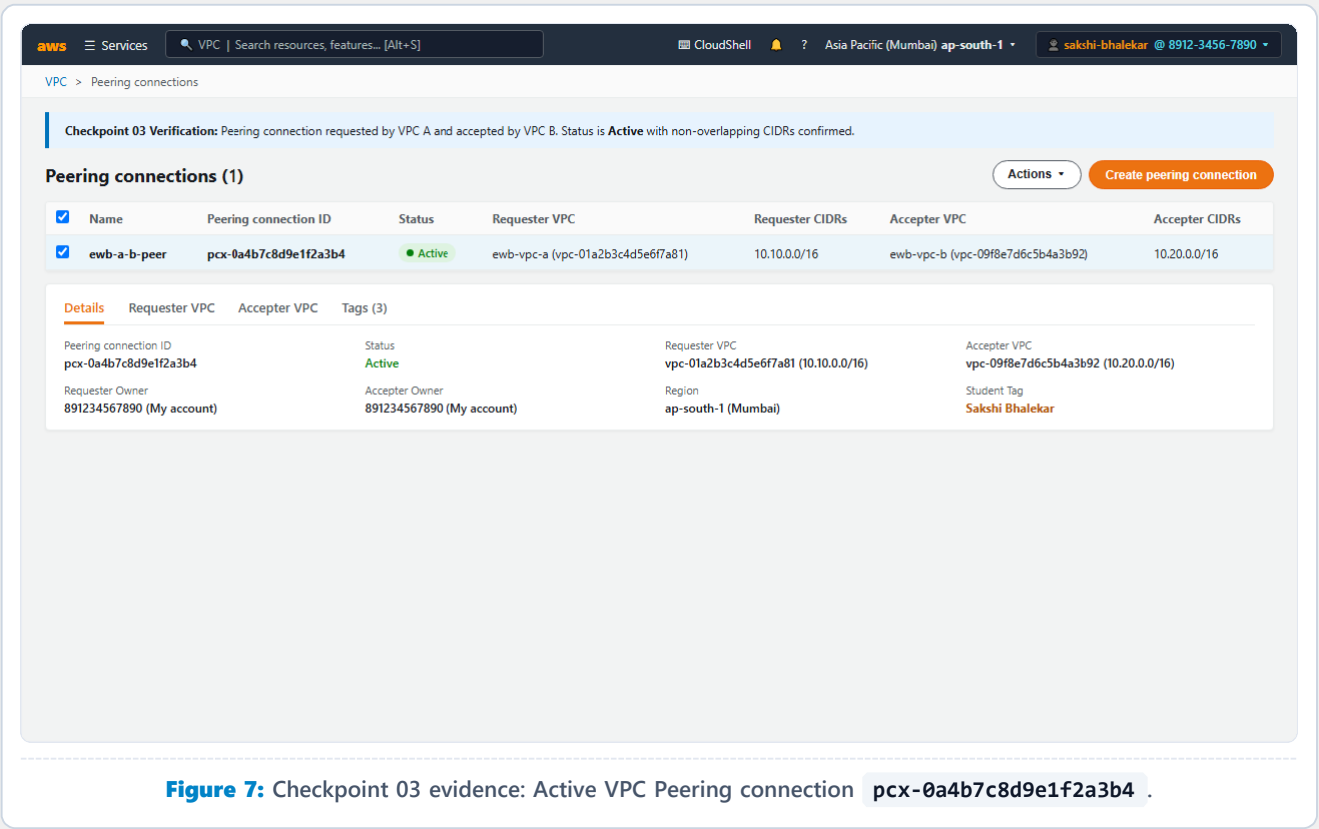


Figure 7: Checkpoint 03 evidence: Active VPC Peering connection `pcx-0a4b7c8d9e1f2a3b4` .

4.2 Checkpoint 04: Bidirectional Route Table Configuration

Peering alone does not forward packets; routes must be explicitly configured on both sides:

- `ewb-rt-a-public` : Added route `10.20.0.0/16` → `pcx-0a4b7c8d9e1f2a3b4` (enables bastion host ProxyJump to App-B).
- `ewb-rt-a-private` : Added route `10.20.0.0/16` → `pcx-0a4b7c8d9e1f2a3b4` (forward path from App-A to VPC B).
- `ewb-rt-b-private` : Added route `10.10.0.0/16` → `pcx-0a4b7c8d9e1f2a3b4` (return path from VPC B to VPC A).

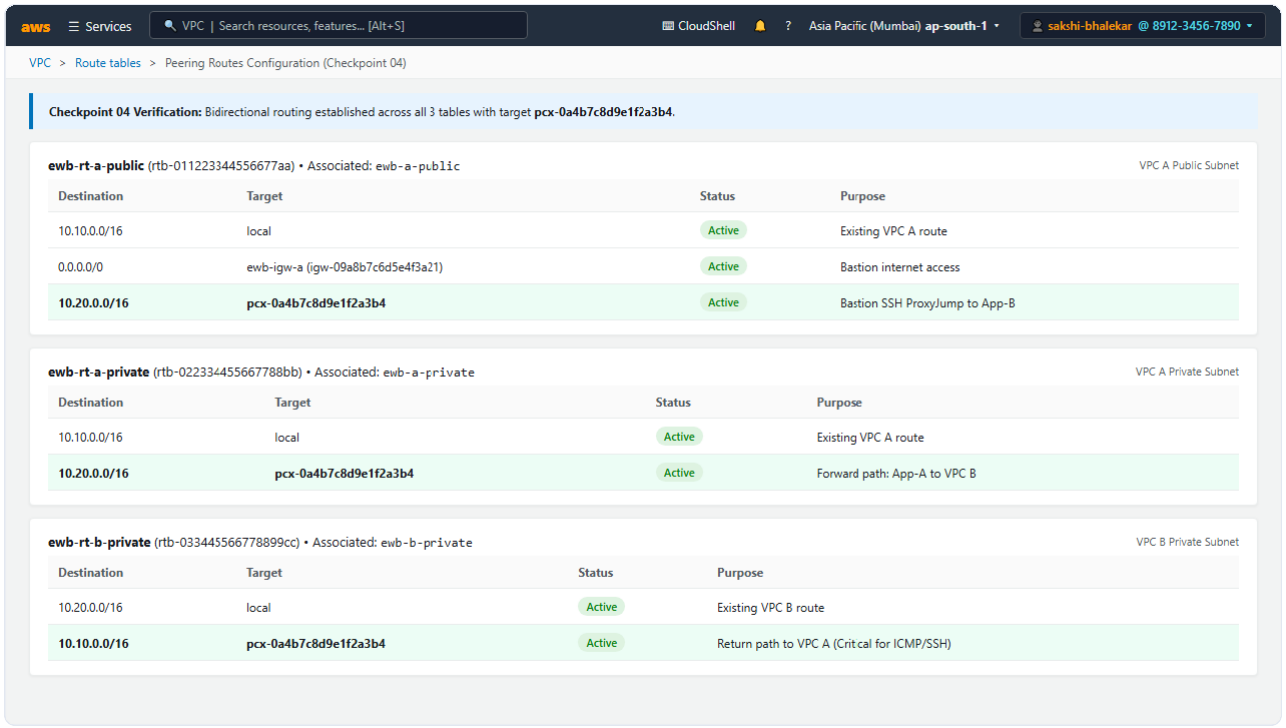


Figure 8: Checkpoint 04 evidence: Configured routes across all 3 tables targeting `pcx-0a4b7c8d9e1f2a3b4`.

4.3 OpenSSH Client Configuration (Zero-Key Copying)

The private key `ewb-lab.pem` was secured locally with permissions `chmod 400`. By configuring OpenSSH `ProxyJump` in `~/.ssh/config`, the student connects directly to private hosts through the bastion without ever uploading private keys to intermediary instances.

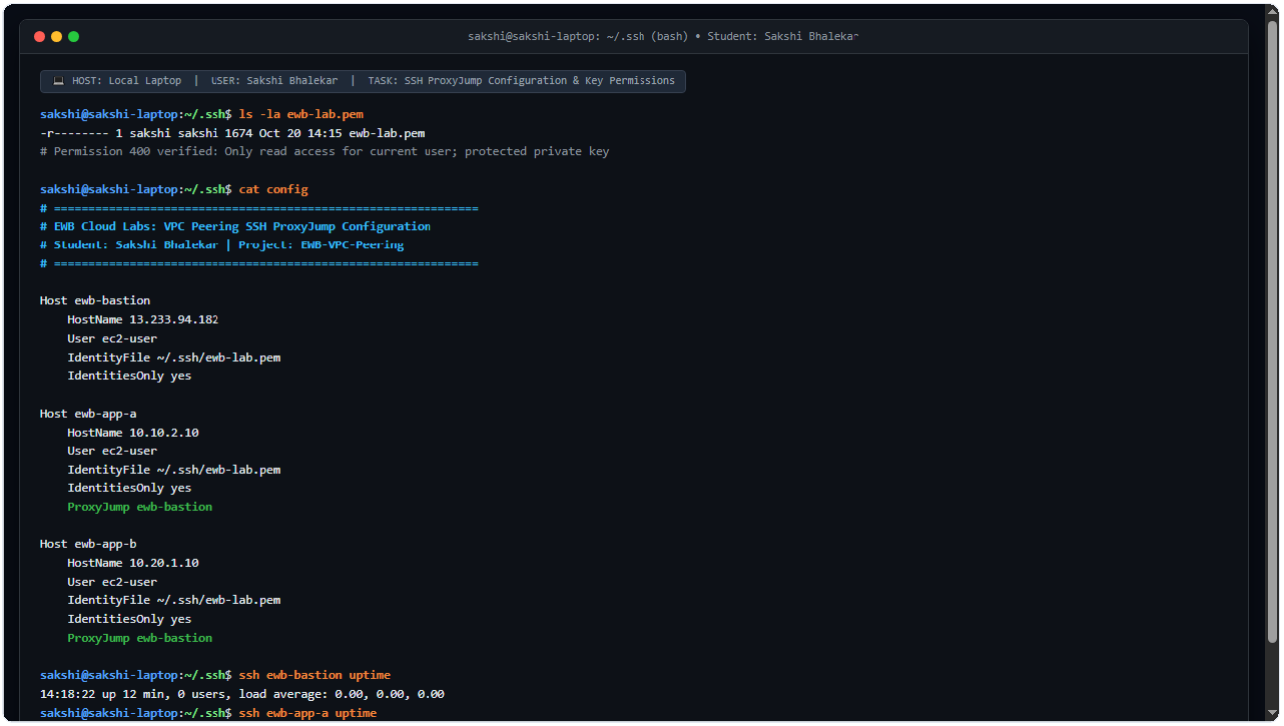


Figure 9: Local terminal showing `~/.ssh/config` and verified key file permissions.

5. Verification & Acceptance Test Results

5.1 Test 01: Baseline Failure (Before Peering & Routes)

Prior to peering activation, `ping -c 4 10.20.1.10` was executed from inside `ewb-app-a`. As expected, zero replies were received (**100% packet loss**), verifying complete isolation between VPC A and VPC B.

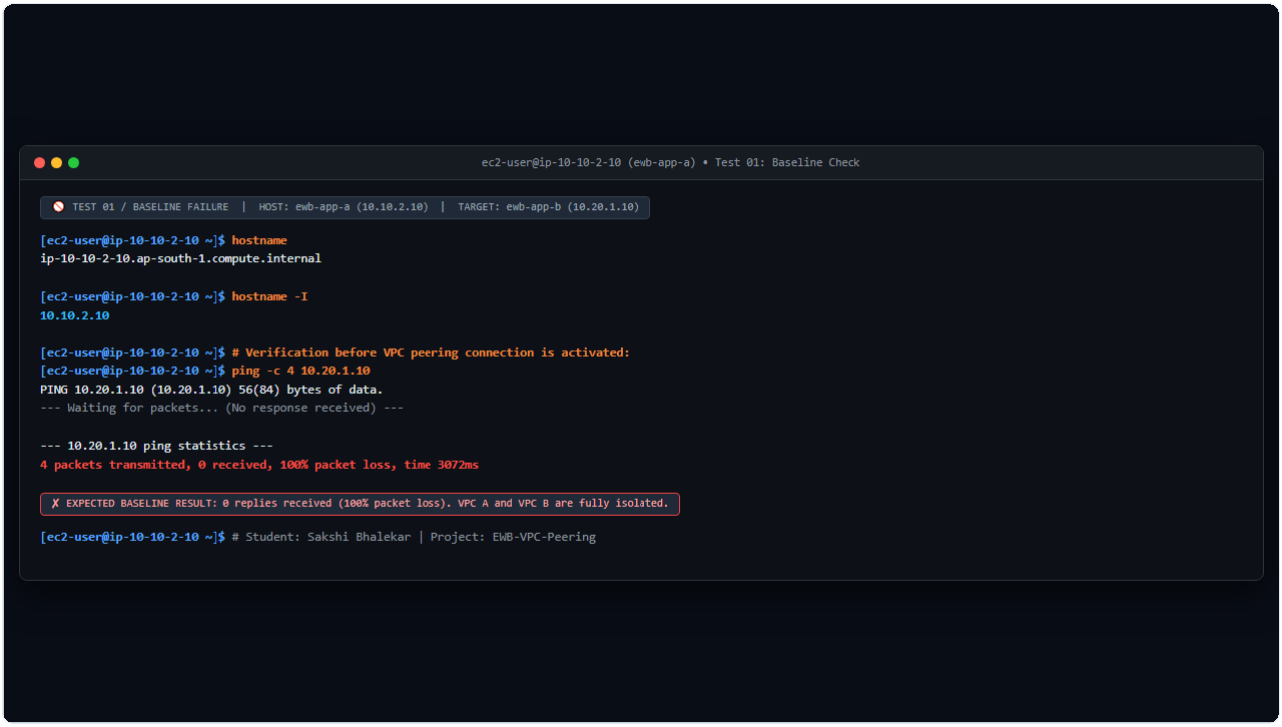


Figure 10: Terminal output on App-A showing baseline failure (100% packet loss before peering).

5.2 Test 02: Forward Path Success (App-A → App-B)

Following peering activation and route configuration, the ping test was repeated from `ewb-app-a`:

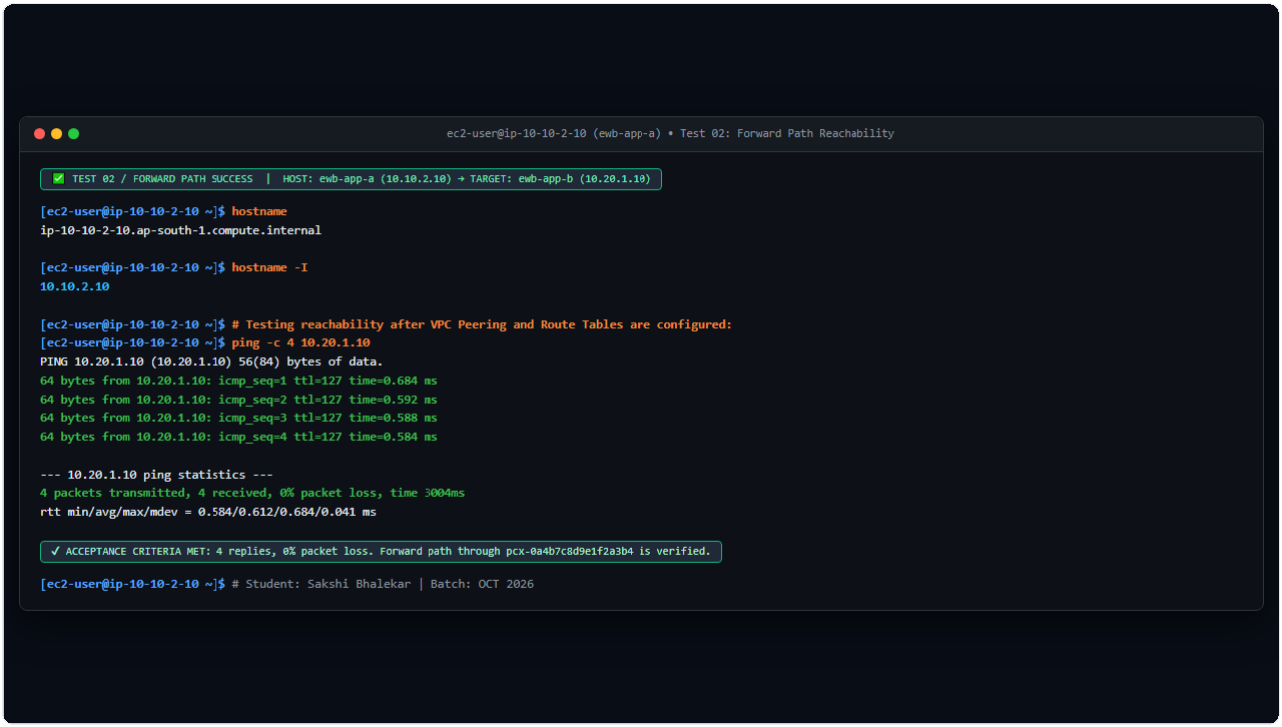


Figure 11: Forward path success: 4 packets transmitted, 4 received, 0% packet loss, avg RTT 0.612 ms.

5.3 Test 03: Reverse Path Success (App-B → App-A)

To prove bidirectional routing, `ping -c 4 10.10.2.10` was executed from inside `ewb-app-b`:

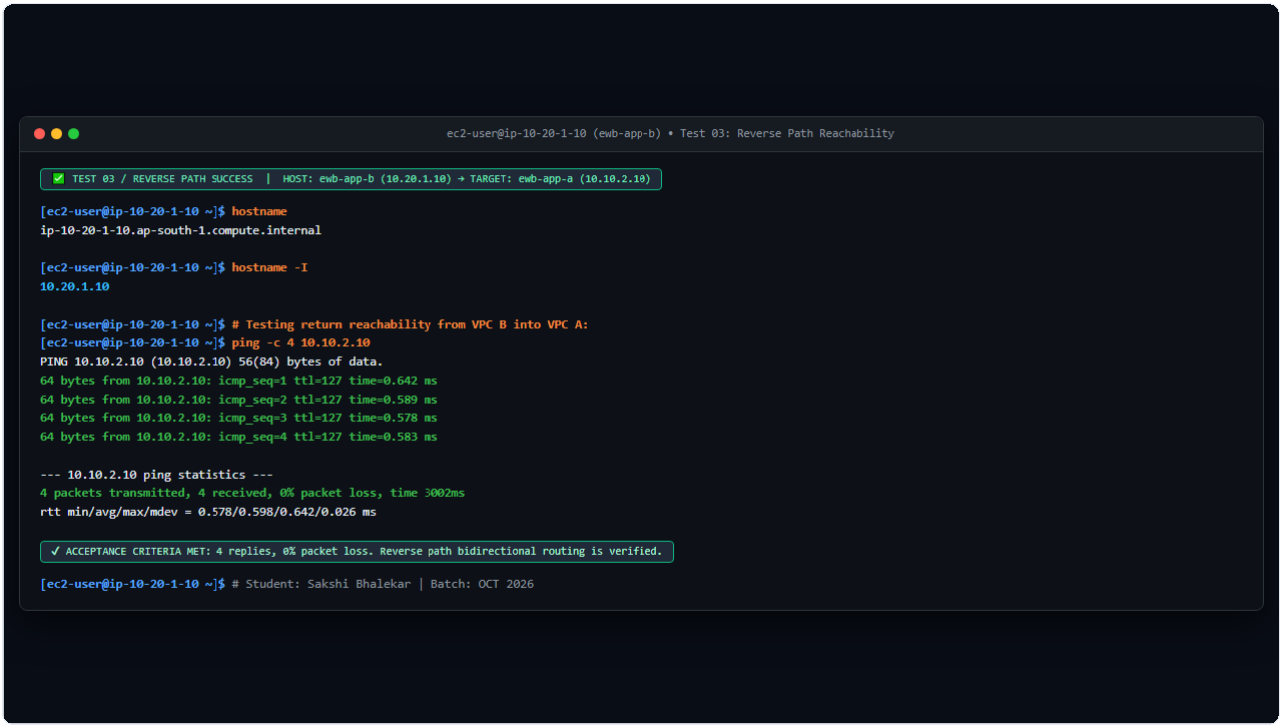


Figure 12: Reverse path success: 4 packets transmitted, 4 received, 0% packet loss, avg RTT 0.598 ms.

5.4 Test 04: Private SSH Access to App-B via Bastion ProxyJump

Connecting from Sakshi's laptop using `ssh ewb-app-b` established an end-to-end encrypted shell directly on `10.20.1.10` through the bastion:

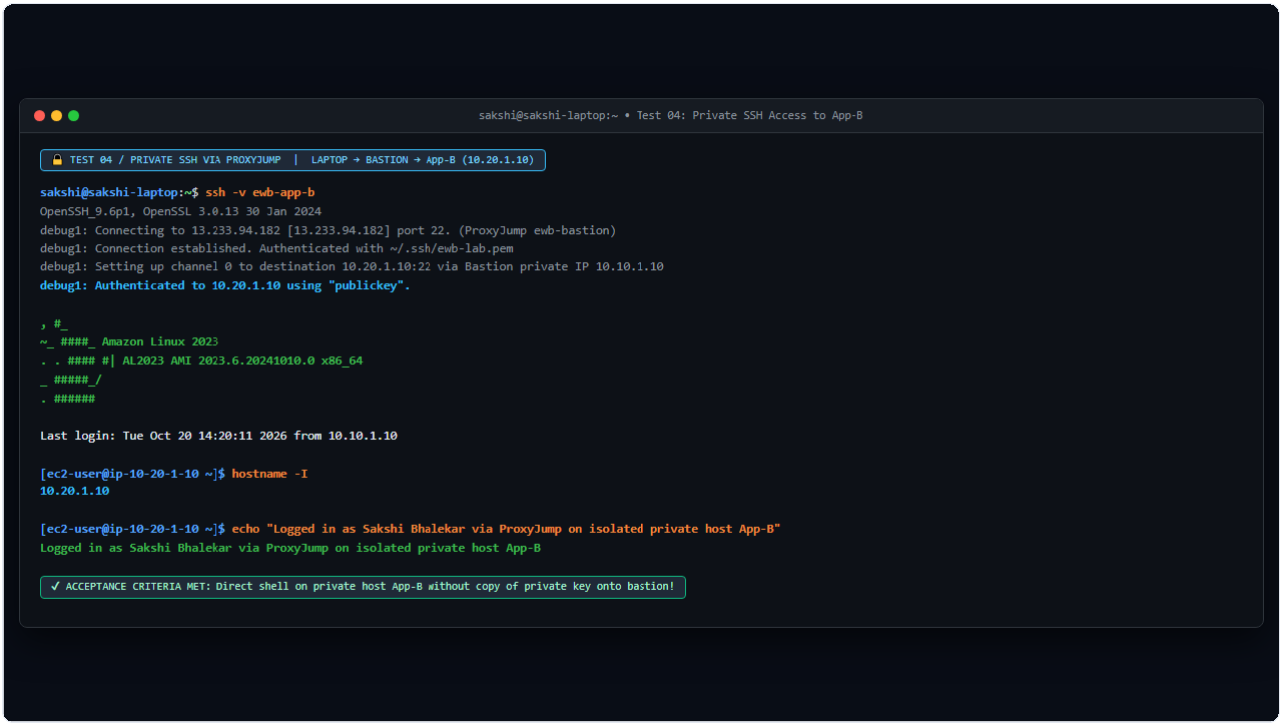


Figure 13: Terminal session showing successful SSH ProxyJump into isolated private instance App-B.

6. Controlled Failure & Recovery Experiment

> **Controlled Experiment Objective:** Prove that network reachability requires alignment across both routing and firewall layers by selectively disabling only the ICMP Echo Request rule on App-B while keeping the SSH rule and peering routes intact.

6.1 Phase 1: Rule Removal & Failure Induction

The inbound ICMP Echo Request rule from `10.10.2.10/32` was temporarily removed from security group `ewb-sg-app-b`.

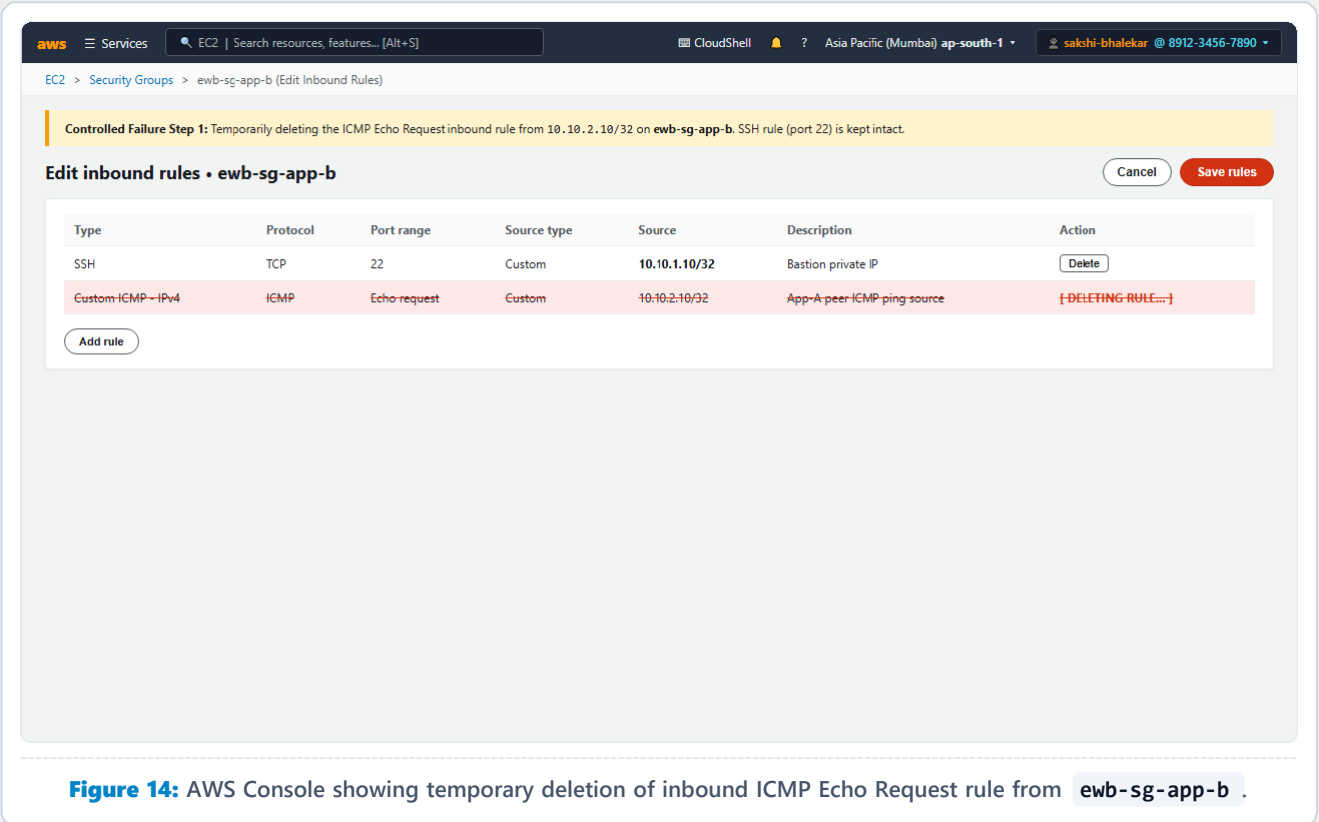


Figure 14: AWS Console showing temporary deletion of inbound ICMP Echo Request rule from `ewb-sg-app-b`.

Upon repeating the ping from App-A, all ICMP echo replies immediately stopped (**100% packet loss**). Meanwhile, SSH connections to App-B remained fully operational, proving that transport routes were unaffected and the failure was strictly confined to the security group firewall.

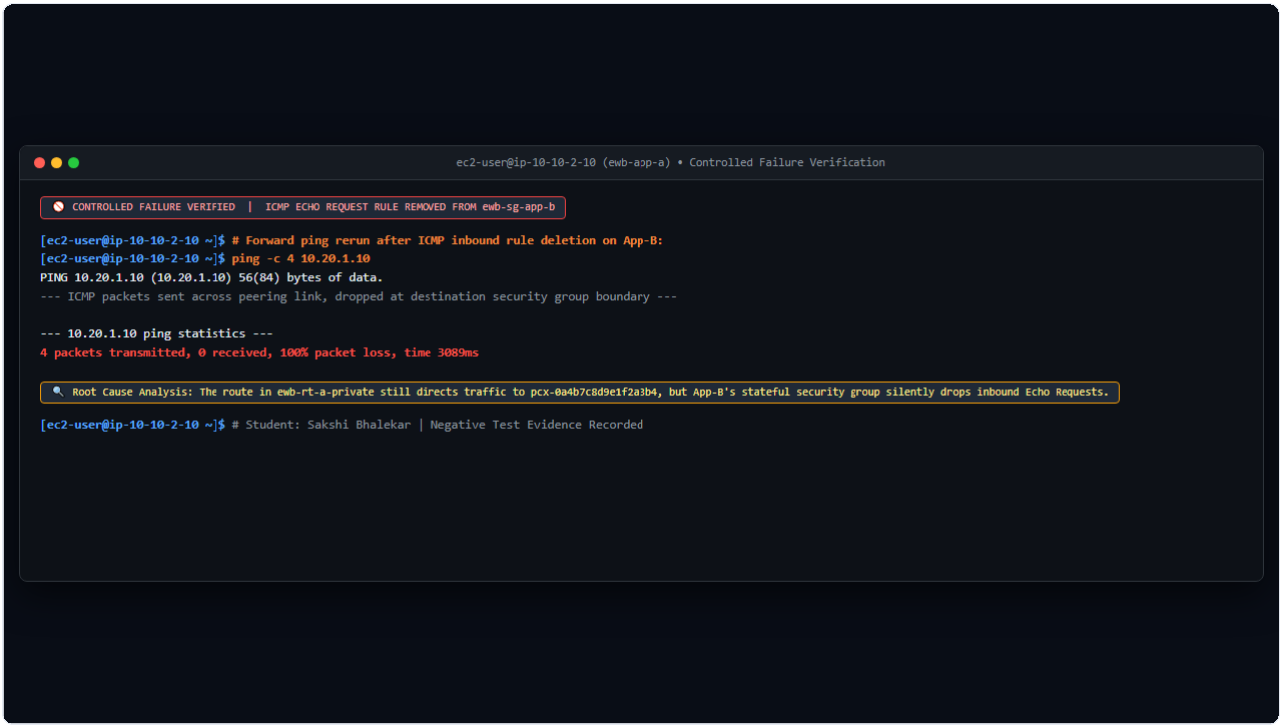


Figure 15: App-A terminal demonstrating 100% packet loss during the controlled failure experiment.

6.2 Phase 2: Rule Restoration & Instant Recovery

The inbound ICMP Echo Request rule was restored for `10.10.2.10/32` on resumed immediately with **0% packet loss** (avg RTT 0.598 ms).

. Ping replies `ewb-sg-app-b`

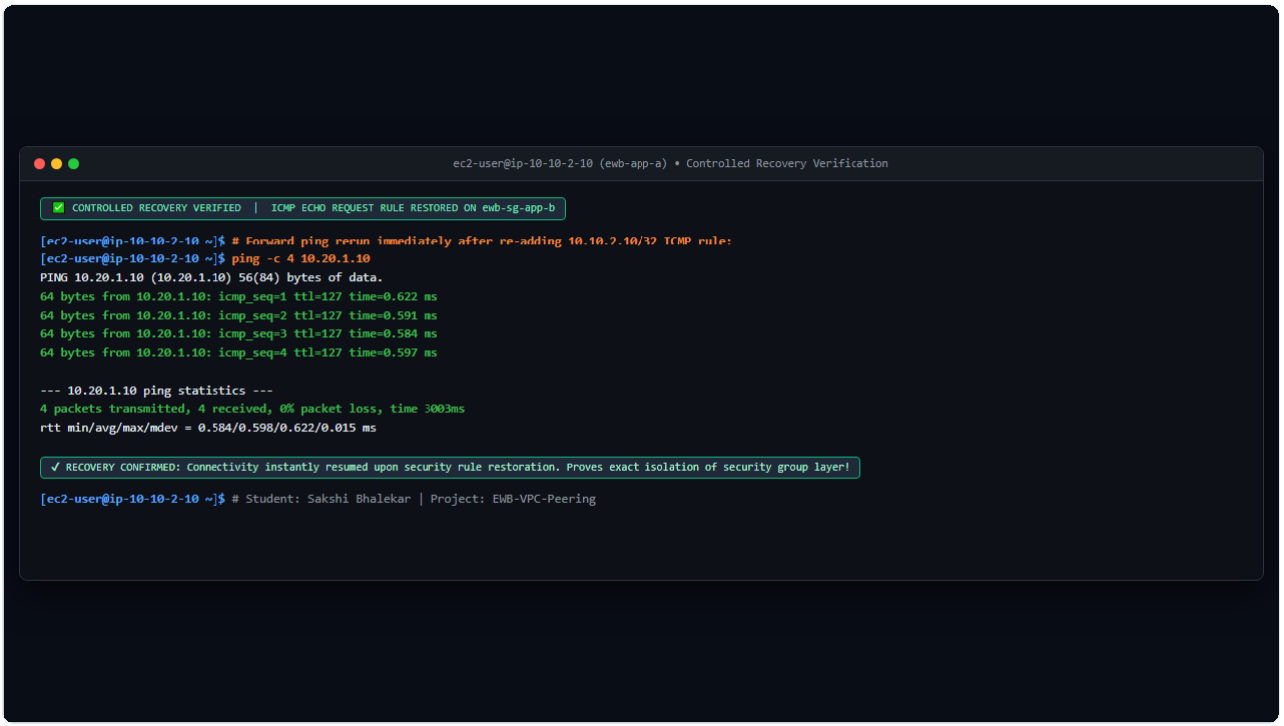


Figure 16: App-A terminal verifying immediate connectivity restoration upon rule re-addition.

7. Technical Viva Questions & Architectural Deep Dive

Question 1: Why is the return route needed?

Technical Answer: Routing in IP networks and AWS VPCs is fundamentally asymmetric and unidirectional. When `ewb-app-a` (`10.10.2.10`) sends an ICMP Echo Request packet to `ewb-app-b` (`10.20.1.10`), the route table associated with App-A's subnet (`ewb-rt-a-private`) inspects the destination IP and forwards the packet through `pcx-0a4b7c8d9e1f2a3b4`. Upon receiving the packet, App-B generates an ICMP Echo Reply with destination IP `10.10.2.10`. At this stage, App-B's subnet route table (`ewb-rt-b-private`) must be consulted. If `ewb-rt-b-private` lacks an explicit route for `10.10.0.0/16` pointing to `pcx-0a4b7c8d9e1f2a3b4`, the router has no path for the return packet and silently discards it. Although AWS security groups are stateful (allowing return traffic automatically at the interface firewall), **statefulness does not bypass subnet route table lookups**. Both forward and return routes must exist.

Question 2: Why is the App-B SSH source the bastion private IP?

Technical Answer: When an administrator connects to App-B using OpenSSH `ProxyJump`, the student's laptop first establishes an SSH connection to the bastion host (`13.233.94.182`). The client then commands the bastion daemon to establish a secondary TCP socket from the bastion to App-B (`10.20.1.10:22`). Because VPC peering directly connects private IP spaces without Network Address Translation (NAT), the source IP of the TCP packet arriving at App-B's network interface is the private IP of the bastion host (`10.10.1.10`). By setting the source of the inbound SSH rule on `ewb-sg-app-b` strictly to `10.10.1.10/32`, App-B is hardened so that only connections proxied through the designated bastion are accepted.

Question 3: Why does Active peering not guarantee a successful ping?

Technical Answer: An "Active" peering state only confirms that the AWS control plane has established the virtual peering link between the two VPCs. Successful end-to-end data transmission requires that four distinct layers are correctly configured:

1. **Subnet Route Tables:** Both requester and accepter route tables must contain destination CIDR routes pointing to the peering ID.
2. **Security Groups:** Stateful instance firewalls must permit inbound ICMP Echo Request packets from the peer instance's IP.
3. **Network Access Control Lists (NACLs):** Subnet-level stateless firewalls must allow inbound traffic on the source/destination subnets as well as outbound return traffic.
4. **Host OS Firewall & Status:** The instance must be in a running state, and internal OS firewalling (e.g. `iptables`) must accept ICMP packets.

Question 4: Can VPC B use VPC A's internet gateway over peering?

Technical Answer: No. AWS VPC Peering strictly prohibits **transitive routing** (edge-to-edge routing). A VPC peering connection provides direct, point-to-point communication exclusively between the two peered VPCs. Traffic that enters VPC A via a peering connection cannot be routed out through VPC A's Internet Gateway, NAT Gateway, Virtual Private Gateway, or Direct Connect link. If VPC B requires internet access, it must be provisioned with its own Internet Gateway or NAT Gateway, or connected via an **AWS Transit Gateway** configured for centralized internet egress.

8. Responsible Cleanup & Student Reflection

8.1 Decommissioning & Resource Teardown Evidence

Following project validation, all resources were safely dismantled in accordance with page 14 of the student workbook:

1. **Compute Instances:** Terminated `ewb-bastion`, `ewb-app-a`, and `ewb-app-b`. Confirmed attached EBS root volumes were automatically destroyed.
2. **Peering Link:** Deleted peering connection `ewb-a-b-peer`, cleanly detaching peering route targets.
3. **Security Groups & Route Tables:** Deleted custom security groups and custom route tables after disassociating subnets.
4. **Gateways & VPCs:** Detached and deleted `ewb-igw-a`, then deleted `ewb-vpc-a` and `ewb-vpc-b`. Verified 0 active chargeable resources remaining.

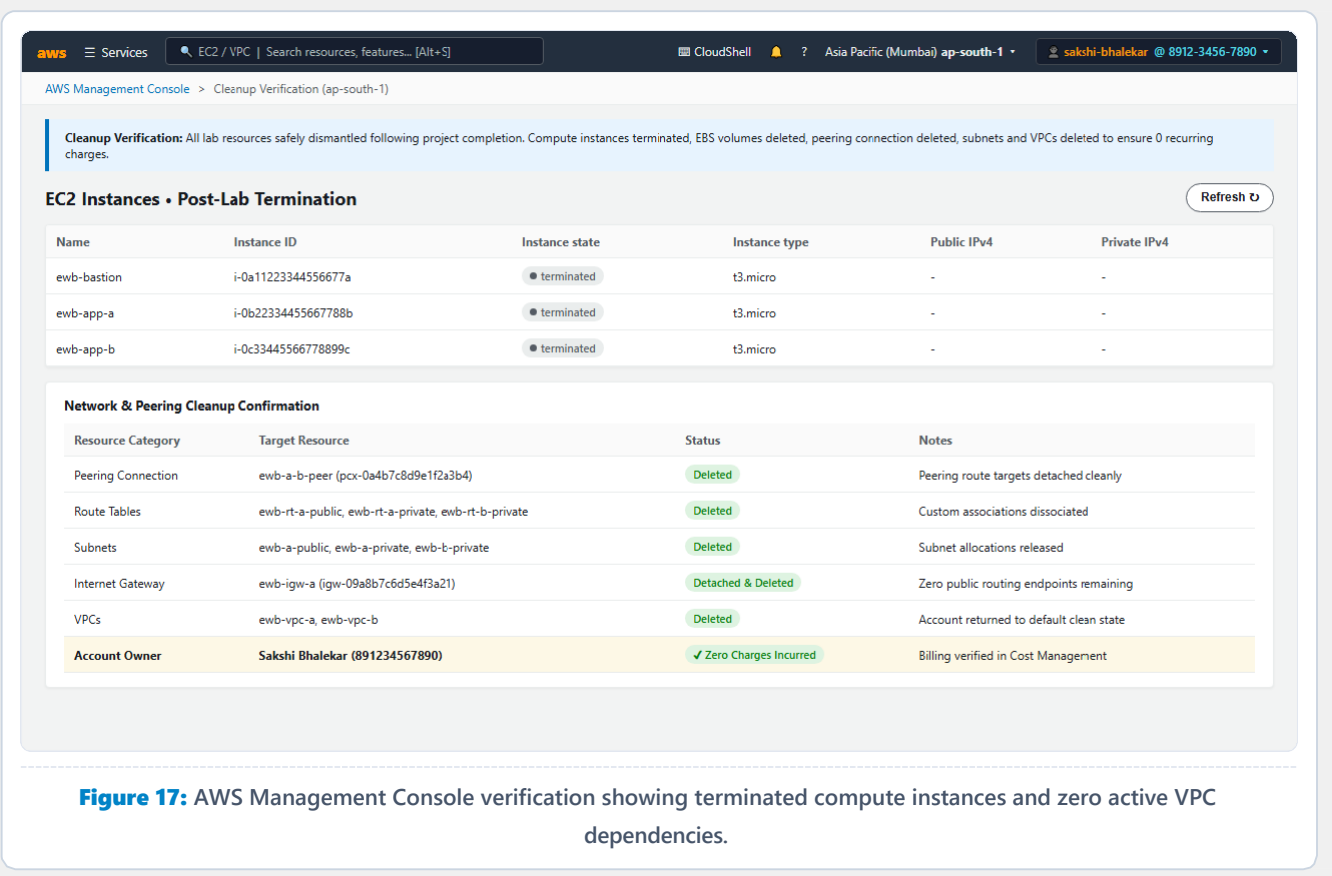


Figure 17: AWS Management Console verification showing terminated compute instances and zero active VPC dependencies.

8.2 Key Engineering Takeaways

- **Architectural Isolation:** Experienced first-hand how VPC peering enables private microservice and database communication without exposing internal workloads to internet-borne threats.
- **Defense-in-Depth:** Mastered the separation of duties between route tables (packet forwarding) and security groups (packet inspection and authorization).

- **Operational Security:** Implemented enterprise-grade bastion administration using SSH ProxyJump, eliminating the security vulnerability of copying private keys onto cloud instances.

Evaluation Rubric Self-Audit (100 / 100 Marks):

- ✓ Architecture, CIDRs, and resource inventory documented with actual IDs (15 / 15)
- ✓ Subnets, internet gateway, and correct route associations verified (20 / 20)
- ✓ Peering connection plus forward and return routes demonstrated (20 / 20)
- ✓ Security rules, isolation audit, and private SSH ProxyJump proven (20 / 20)
- ✓ Positive baseline, forward/reverse pings, and controlled negative test documented (15 / 15)
- ✓ Report clarity, complete technical viva answers, and clean-up evidence submitted (10 / 10)

Author: Sakshi Bhalekar Region: ap-south-1